ChatGPT

Executive Summary

This document specifies a **Dynamic Pricing & Demand Intelligence System (MERN)** for enterprise usage. It models the price–demand relationship and generates revenue-optimizing price recommendations. The system ingests historical sales data (CSV), fits a demand curve (e.g. linear regression $Q = a - bP$ [1]), simulates demand/revenue for hypothetical prices, and finds the optimal price. An interactive React dashboard (using **Recharts** for charts [2] and **Tailwind v4 + Shadcn UI** components [3] [4]) visualizes key metrics. Primary domain: **Revenue Management / Price Analytics** (microeconomics) [5] [6]. Target users are business analysts and pricing managers. Success means the system accurately ingests data, computes demand elasticity, and suggests price changes that maximize revenue (unitary elasticity [6]).

Project Overview

- **Primary Domain:** Business Analytics (Revenue Management, Microeconomics). The system applies price-demand modeling and elasticity to optimize pricing [5] [6].
- **Target Users:** E-commerce or retail pricing analysts, product managers, or operations teams. They use it to set prices and test pricing scenarios.
- **Success Criteria:** Ability to import real sales data, fit a demand model, simulate pricing scenarios, and recommend the price that maximizes revenue. The UI should display clear demand/revenue curves and KPIs.

Feature List

Core Features

- **Data Ingestion:** Upload historical sales data (CSV of price vs. quantity, with optional time or competitor price). This populates the database.
- **Demand Modeling:** Fit a demand curve to data (e.g. linear $Q=a-bP$ regression [1]). Compute parameters and store the model.
- **Price Simulation:** Given an input price (and optional factors like competitor price), compute predicted demand (Q) and revenue ($P \times Q$) using the fitted model.
- **Revenue Optimization:** Scan price range to find the price that maximizes revenue (derivative of $R(P)=P \cdot Q(P)$ set to zero, or unit-elastic point [6]).
- **Visualization Dashboard:** Interactive UI with charts: Demand vs. Price and Revenue vs. Price curves, KPI cards (current revenue, demand, optimal price).

Enhancement Features

- **Break-even / Profit Analysis:** Include cost per unit to compute profit (revenue minus cost). Identify break-even price.

- **Price Elasticity Indicator:** Calculate and display elasticity $E=(\Delta Q/Q)/(\Delta P/P)$ [5] at given price. Show if demand is elastic or inelastic.
- **Competitor Price Adjustment:** Allow input of competitor's price; adjust demand prediction accordingly (e.g. reduce demand if competitor's price is lower).
- **Scenario Comparison:** Run multiple simulations side-by-side (e.g. compare current price vs. new price scenario).
- **Multi-Product Support:** Manage and simulate pricing for multiple products.
- **Time-Series Forecasting (Optional):** Add moving-average or regression forecast to adjust baseline demand (e.g., seasonal factor).

Optional Features

- **User Authentication & Roles:** Basic login to restrict access (e.g. Admin, Analyst roles).
- **Automated Data Refresh:** Scheduled ingestion from an external data source (API or database).
- **Alerts/Notifications:** Rule-based alerts (e.g., "Price too elastic, revenue risk" or "New optimal price found").
- **Deployment Config:** Docker container, CI/CD pipeline (GitHub Actions) for production build.

Technical Mapping

Backend APIs

Endpoint	Method	Request JSON	Response JSON	Purpose
`/upload-sales`	POST	`{ file: CSV }`	`{ success: true }`	Parse CSV, save records to **SalesData** collection.
`/fit-model`	POST	`{ productId: ID }` (optional)	`{ a: number, b: number }`	Read sales data, fit $Q=a-bP$ via least squares, store model coefficients (e.g. in **Product** or **Model** collection).

Endpoint	Method	Request JSON	Response JSON	Purpose
`/simulate`	POST	`{ productId: ID, price: number, competitorPrice?: number }`	`{ demand: number, revenue: number, profit?: number }`	Compute predicted demand and revenue (and profit if cost known) using model.
`/optimize`	POST	`{ productId: ID, range: [min, max], step: number }`	`{ optimalPrice: number, maxRevenue: number, curve: [ {price, revenue} ] }`	Sweep price range, simulate revenue, find max. Return optimum and revenue curve data.
`/products`	GET/ POST	(Create: `{name,cost,basePrice,...}`)	(List: `[{id,name,cost,...}]`)	CRUD for products metadata (name, cost, etc.).
`/elasticity` (opt)	GET	`{ price1, price2, demand1, demand2 }`	`{ elasticity: number }`	Compute E=(%ΔQ/ %ΔP) between two price–demand points (for reference).

Input/output JSON structures are examples; adjust fields as needed.

MongoDB Schemas

Collection	Fields (type)	Description
Product	`{ _id: ObjectId, name: string, basePrice: number, cost: number, ... }`	Basic product info and cost data.

Collection	Fields (type)	Description
SalesData	`{ _id: ObjectId, productId: ObjectId (ref), price: number, quantity: number, competitorPrice?: number, date?: Date }`	Historical sales records. Each row holds one sale or aggregated daily data.
Model	`{ productId: ObjectId, a: number, b: number, lastUpdated: Date }`	Fitted demand model parameters ($Q = a - bP$) for each product.
Simulation	`{ _id: ObjectId, productId: ObjectId, inputPrice: number, demand: number, revenue: number, profit?: number, timestamp: Date }`	Log of simulations run (optional). Useful for audit/ history.
User (opt)	`{ username: string, hashedPassword: string, role: string }`	(If auth used) user credentials and roles.

(MongoDB uses BSON (binary JSON) internally [7], so collections store data in JSON-like documents.)

Frontend Components (React with Tailwind + Shadcn)

Component	Props / Usage	Behavior	Chart Type
FileUploader	`onUpload(file)`	Renders a file input/button, uploads CSV to `/upload-sales`.	–
Dashboard	none (root page)	Layout with sidebar/navigation. Shows KPI cards and charts.	–
PriceInput	`price, onChange`	Numeric input or slider for setting test price.	–
DemandChart	`data: [{price, demand}]`	Line chart (Recharts) of Price vs. Demand.	LineChart (XY)
RevenueChart	`data: [{price, revenue}]`	Line chart of Price vs. Revenue.	LineChart (XY)
KPIcard	`label, value`	Displays a key metric (e.g. “Max Revenue”).	–
ProductSelector	`products, onSelect`	Dropdown/select to choose product (if multi-product).	–
ScenarioPanel	`scenarios`	Shows multiple simulation results side-by-side for comparison.	–
ElasticityBadge	`elasticity`	Displays elasticity value and “Elastic/ Inelastic” status.	–

Each visual component uses **Tailwind CSS** classes (e.g. `flex`, `container`, `text-2xl`) and Shadcn UI primitives [3] [4] for styling and layout (e.g. `<Card>`, `<Sidebar>`, `<Slider>`). For example, `<LineChart>` from Recharts is embedded inside a responsive container.

Algorithms and Formulas

- **Linear Demand Model:** We assume a linear demand $Q = a - b \cdot P$ [1]. Fit coefficients a, b using ordinary least squares on (price, quantity) data.
- **Least-Squares Regression:** Solve for a, b by minimizing $\Sigma (Q_i - (a - b \cdot P_i))^2$. (Formulas: $b = [N \Sigma P_i Q_i - \Sigma P_i \Sigma Q_i] / [N \Sigma P_i^2 - (\Sigma P_i)^2]$; $a = (\Sigma Q_i + b \Sigma P_i)/N$.)
- **Price Elasticity:** $E = (\%\Delta Q) / (\%\Delta P) = (\Delta Q/Q) / (\Delta P/P)$ [5]. At price P, elasticity = $b \cdot (P/Q)$ for linear model (negative value typically).
- **Revenue:** $R(P) = P \cdot Q(P) = P \cdot (a - bP)$. To maximize revenue, set derivative $dR/dP = a - 2bP = 0 \Rightarrow P = a/(2b)$ [6] (this yields $|E| = 1$).
- **Break-Even / Profit:** If cost per unit c is known, profit = $(P - c) \cdot Q(P)$. Break-even at $P = c$.
- **Scenario (Competitor Effect):** A simple adjustment: if competitor price Pc < P, reduce predicted demand by factor (e.g. $Q\_adj = Q \cdot (1 - discountFactor(P - Pc)/P)$*).

Core Endpoint Pseudocode

```
// /upload-sales (POST): parse CSV and save to SalesData
function uploadSales(csvFile):
    rows = parseCSV(csvFile)
    for row in rows:
        // e.g. {productId, price, quantity, competitorPrice, date}
        SalesData.insert({
            productId: row.productId,
            price: row.price,
            quantity: row.quantity,
            competitorPrice: row.competitorPrice,
            date: row.date
        })
    return { success: true }

// /fit-model (POST): fit linear demand to saved data
function fitModel({ productId }):
    data = SalesData.find({ productId })
    // build arrays P and Q
    N = data.length
    sumP = sum(data.price), sumQ = sum(data.quantity)
    sumPQ = sum(data.price * data.quantity)
    sumP2 = sum(data.price^2)
    // least-squares linear fit Q = a - bP
    b = (N*sumPQ - sumP*sumQ) / (N*sumP2 - sumP^2)
    a = (sumQ + b*sumP) / N
    // store model parameters
```

```
    Model.upsert({ productId }, { a: a, b: b, lastUpdated: now })
    return { a: a, b: b }

// /simulate (POST): compute demand and revenue for given price
function simulate({ productId, price, competitorPrice }):
    model = Model.findOne({ productId })
    a = model.a; b = model.b
    // adjust if competitorPrice given (optional heuristic)
    adjFactor = 1
    if (competitorPrice) {
        // example: if competitor price < our price, reduce demand by factor
        adjFactor = 1 - k * max(0, (price - competitorPrice)/price)
    }
    demand = max(0, (a - b*price) * adjFactor)
    revenue = price * demand
    profit = revenue - (Product.cost * demand)  // if cost known
    // Optionally log simulation
    Simulation.insert({ productId, inputPrice: price, demand, revenue, profit,
timestamp: now })
    return { demand, revenue, profit }

// /optimize (POST): find price with max revenue
function optimize({ productId, minPrice, maxPrice, step }):
    best = { price: minPrice, revenue: 0 }
    curveData = []
    for (p = minPrice; p <= maxPrice; p += step):
        { demand, revenue } = simulate({ productId, price: p })
        curveData.push({ price: p, revenue })
        if (revenue > best.revenue) best = { price: p, revenue }
    return { optimalPrice: best.price, maxRevenue: best.revenue, curve:
curveData }
```

Implementation Roadmap

8-Day Timeline (MVP Focus):

- **Day 1:** Set up project skeleton (Node/Express backend, MongoDB connection, React/Tailwind frontend). Define data models (Mongo schemas) and basic routes.
- **Day 2:** Implement CSV upload (`/upload-sales`): parse file, insert records into **SalesData**. Add simple UI component (`FileUploader`).
- **Day 3:** Implement demand fitting (`/fit-model`): run linear regression on sales data, store coefficients. Test with sample data.
- **Day 4:** Implement simulation (`/simulate`) and optimization (`/optimize`) logic. Validate with known linear model (e.g. synthetic data).
- **Day 5:** Build main UI layout (Dashboard page with sidebar, selectors, KPI cards) using Tailwind and Shadcn components [3] [4] .

- **Day 6:** Integrate charts: use Recharts `<LineChart>` for demand and revenue curves [8]. Hook up inputs (price slider/input) to `/simulate` and update charts/KPIs.
- **Day 7:** Add optimization results: display recommended price, run `/optimize` and plot revenue curve. Implement break-even/profit display.
- **Day 8:** Testing and polish: fix layout/UX issues, handle edge cases (no data), refine styling. Prepare demo and documentation.

4-Week Roadmap (Extended):

- **Week 1 (Core):** Same as days 1–4 above plus robust error handling, data validation.
- **Week 2 (UI & Visualization):** Complete dashboard, add Shadcn components (Cards, Sidebars, Sliders) [3] [4]. Responsive design, tooltips.
- **Week 3 (Enhancements):** Implement elasticity indicator, competitor-price effect, multi-product selection, scenario comparison. Add user auth if needed.
- **Week 4 (Optimization & Deployment):** Add advanced analytics (forecasts, what-if scenarios), containerize app (Docker), set up CI/CD pipeline, final testing and documentation.

Testing Checklist

- **Data Ingestion:** Uploading CSV with known price–quantity pairs correctly populates the **SalesData** collection.
- **Model Fitting:** `a` and `b` match expected values for test data (e.g., linear data). Check `Q=a−bP` fits points.
- **Simulation:** For a given price, computed demand/revenue follow the model (e.g., if $Q=a-bP$, verify outputs).
- **Optimization:** For a simple parabola revenue, `/optimize` finds correct peak (compare to analytic P=a/(2b)).
- **Charts:** Demand and Revenue charts correctly plot data (monotonic decline/increase). Axes labels and units correct.
- **UI Interactions:** Sliders and inputs update charts/KPIs in real time. No console errors.
- **Edge Cases:** Handling of missing data, zero demand, negative prices, etc. Proper error messages.
- **Security:** (If implemented) API endpoints reject unauthorized requests.

Dummy Dataset Example

Columns and sample row for input CSV:

productId	price	quantity	competitorPrice	date
1	10	100	12	2026-01-01

- *productId:* (optional) identifier linking to a product.
- *price:* historical price charged.
- *quantity:* units sold at that price.
- *competitorPrice:* price of a rival product (optional).
- *date:* timestamp of sale (optional for time-series).

This row means at price $10 (vs competitor at $12) on Jan 1 2026, sold 100 units.

---

1 MTBCH002.QXD.13008461
https://www.unm.edu/~sarchamb/pindyck_micro06_text_02.pdf

2 8 GitHub - recharts/recharts: Redefined chart library built with React and D3 · GitHub
https://github.com/recharts/recharts

3 4 Tailwind v4 - shadcn/ui
https://ui.shadcn.com/docs/tailwind-v4

5 6 Price elasticity of demand - Wikipedia
https://en.wikipedia.org/wiki/Price_elasticity_of_demand

7 JSON And BSON | MongoDB
https://www.mongodb.com/resources/basics/json-and-bson